Chapitre 6 - Modélisation de l’interaction

CAMELEON : Un framework de conception basé modèle 2

Exprimer des solutions d’UI à un niveau d’abstraction élevé (indépendant des modalités d’interaction) 3

IHM Fils de fer 3

Concept Map 5

Exprimer des solutions d’UI à un niveau de concrétisation suffisant (dépendant des modalité d’interaction) 6

La répartition selon la plateforme : la Meta-IHM 6

Le comportement dynamique : les modèles de dialogue 7

Les automates 7

Les machines à états hiérarchiques (Statecharts pour l’IHM) 8

UAN 9

Représentations des aspects présentation 10

Le placement des concepts : Wireframe 10

Le graphique : la charte graphique 11

Liens entre la présentation et le dynamique : La maquette 11

En conception, les modèles peuvent être utilisés pour comprendre le problème (comme les modèles de tâches par exemple) ou pour exprimer (ou réfléchir) une solution. Ce chapitre s'adresse à ces seconds.

CAMELEON : Un framework de conception basé modèle

Un framework de conception a été entièrement conçu pour exprimer l'intérêt et l'utilisation de ces différents modèles. Le framework Caméléon (Figure 1) propose de ne concevoir un système interactif adapté au contexte qu'en s'appuyant sur des modèles (approche basée modèles).

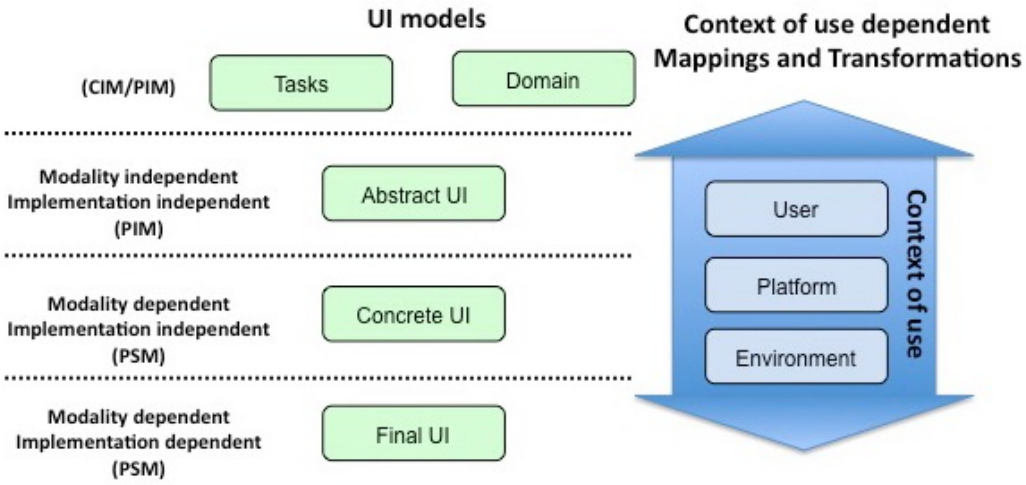


FIGURE 1 : FRAMEWORK CAMELEON (HTTPS://WWW.W3.ORG/COMMUNITY/UAD/WIKI/CAMELEON_REFERENCE_FRAMEWORK)

Ce framework s'appuie sur l'hypothèse que toutes les connaissances nécessaires sont représentées dans les différents modèles (y compris tous les contextes possibles) et que la stricte séparation des préoccupations en conception est possible et souhaitable. Cette approche néglige l'impact des connaissances technologiques sur la définition des enchainements de vues par exemple. Ces deux hypothèses rendent l'application strict du framework tel que présenté rarement possible cependant, l'application de certains paradigmes dans les transformations permet une meilleure traçabilité dans la conception de l'interface.

Le plus haut niveau de préoccupation est celui du domaine et des tâches. Les modèles de ce niveau ont été présentés dans les chapitres 3 et 4. Nous présenterons ici les modèles des deux couches suivante :

- les modèles représentant l'UI a un niveau d'abstraction indépendant des plateformes et des modalités d'interaction (Abstract UI)
- Les modèles représentant l'UI a un niveau de concrétisation suffisant pour prendre en compte les modalités d'interaction

Le niveau le plus concret, dépendant de l'implémentation, est l'interface développée. Le développement d'interfaces utilisateur n'est pas l'objet de ce cours. Cependant, le chapitre 7 abordera des problématiques de conception en lien avec l'implémentation (ex : évaluation de la qualité du code pour l'accessibilité).

Exprimer des solutions d'UI à un niveau d'abstraction élevé (indépendant des modalités d'interaction)

IHM Fils de fer

Hypothèse : une tâche élémentaire interactive a besoin d'un espace pour être réalisée. Cet espace est nommé « espace de travail ». Il sert à délimité une « zone » qui présente les informations à l'utilisateur pour qu'il puisse accomplir sa tâche et les widgets (ou autres éléments d'interaction) qui lui permettre d'agir pour informer le système (pour entrer les informations nécessaires). Une IHM fils de faire permet d'exprimer ces espaces de travail et les relations entre eux en exploitant les connaissances exprimées dans le modèle de tâches.

Exemple : Supposons que nous réalisions une application qui permette de regarder des photos numériques. Le modèle de tâches qui a été défini est celui de la figure 2.

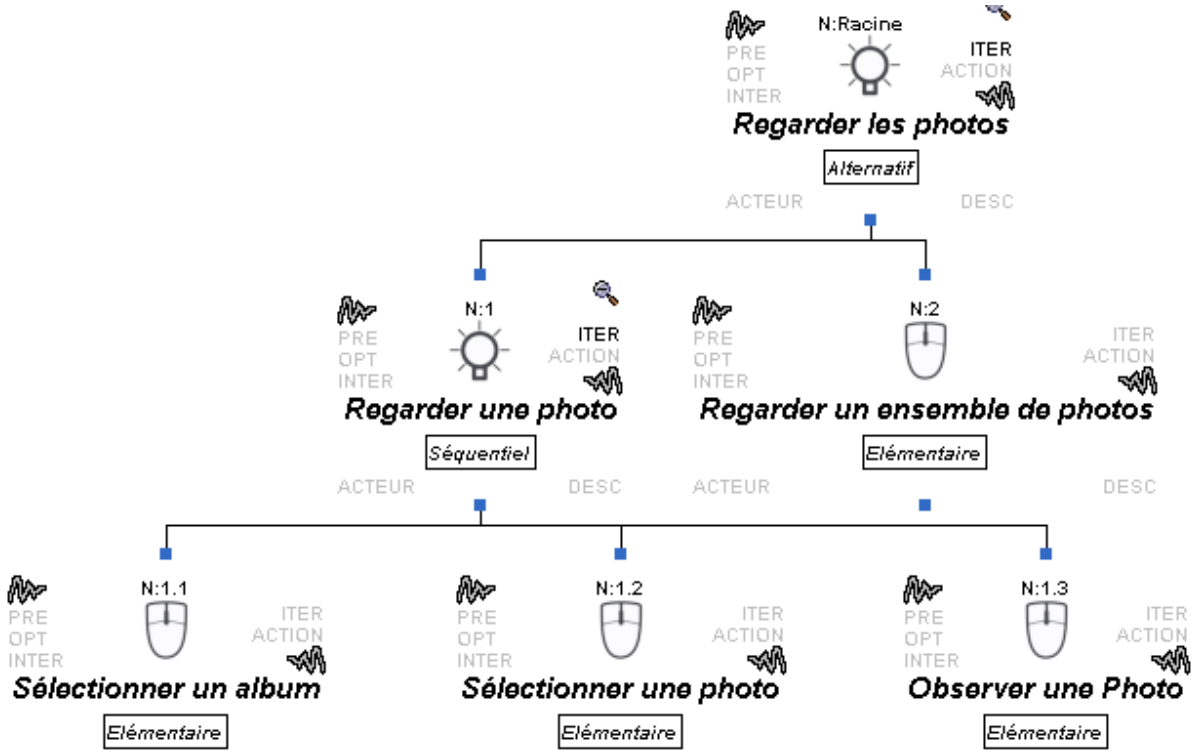


FIGURE 2 : MODÈLE DE TÂCHES « REGARDER LES PHOTOS »

Note : Dans la suite de ce chapitre nous utiliserons cet arbre de tâches cependant il est à noter que dans un cas réel de conception, la tâche « Regarder un ensemble de photos » mériterait une étude plus aboutie (avec la définition de sous tâches à « Regarder un ensemble de photos »).

Dans cet exemple, nous avons 4 tâches élémentaires interactives :

- Sélectionner un album
- Sélectionner une photo
- Observer une photo
- Regarder un ensemble de photo (considérée élémentaire pour l'exemple)

Pour chacune nous créons un espace de travail dédié.

Nom de la tâche	**Nom de l'espace**
Sélectionner un album	Espace de navigation dans les albums
Sélectionner une photo	Espace de sélection d'une photo d'un album
Observer une photo	Espace de visualisation d'une photo
Regarder un ensemble de photo	Espace diaporama

Un espace de travail doit présenter des informations et permettre les entrées de l'utilisateur pour que celui-ci puisse accomplir la tâche pour laquelle l'espace a été créé. Ces informations sont des concepts « métier » issu du modèle de concepts.
Dans la représentation graphique d'une IHM Fil de fer, un espace de travail est présenté comme ceci :

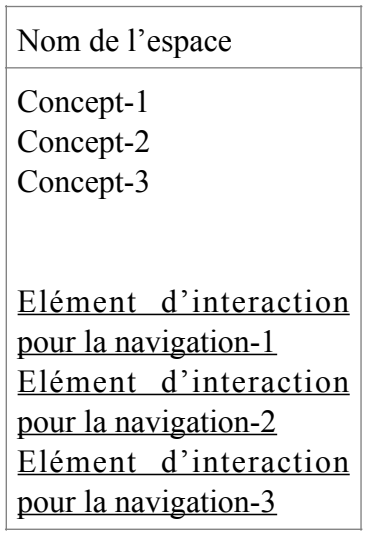


FIGURE 3 : REPRÉSENTATION GRAPHIQUE D'UN ESPACE DE TRAVAIL D'UNE IHM ABSTRAITE

Les concepts à inclure dans les espaces de travail sont les éléments métiers de l'activité. Vous les avez identifiés pendant la réalisation du modèle de tâches (ce sont les objets abstraits dans K-MADe), ou dans une base de données, ou dans un modèle de concepts.
Dans notre exemple, les concepts métiers sont *Album* et *Photo*. Dans les 3 espaces que nous considérons en détails, l'utilisation des concepts est :

- Espace de navigation dans les albums : un ensemble d'albums
- Espace de sélection d'une photo d'un album : un ensemble de photos
- Espace de visualisation d'une photo : une photo

Dans la représentation graphique, un ensemble de Concept-1 est noté [Concept-1] alors qu'un seul élément est noté Concept-1.

La succession possible entre des espaces de travail est directement obtenue à partir des opérateurs d'ordonnancement de l'arbre de tâches et les décorations des tâches. Vous pouvez également exploiter les outils de simulation pour voir comment passer de l'une à l'autre. Graphiquement un élément d'interaction qui permet la navigation entre 2 espaces est représenté par le nom de l'espace destination souligné dans l'espace origine (comme sur la Figure 3). De ce nom souligné part alors une flèche vers l'espace destination.

Les règles à appliquer en fonction des opérateurs d'ordonnancement sont les suivantes :
Ordonnancement séquentiel : Lorsque 2 tâches sont réalisées séquentiellement alors dans l'espace de travail de la première il existe un élément d'interaction qui permet d'accéder à l'espace de travail de la seconde.
Ordonnancement parallèle : Lorsque 2 tâches sont réalisées parallèlement alors les deux espaces de travail sont regroupés en un seul.
Ordonnancement alternatif : Lorsque 2 tâches sont réalisées alternativement alors un espace de travail supplémentaire est ajouté pour permettre le choix par l'utilisateur de la tâche à accomplir. Dans l'espace de travail créé il existe un élément d'interaction qui permet d'accéder à chacun des espaces de travail des tâches alternatives.
Dans l'exemple la tâche *Regarder les photos* ordonnance alternativement les tâches regarder une photo et *Regarder un ensemble de photos*. On crée donc un espace Espace de choix du mode de visualisation de photos qui contient des éléments d'interaction qui permettent d'accéder à l'Espace diaporama et à l'espace de la première tâche réalisée pour accomplir la tâche *Regarder une photo* (=> Sélectionner un album).
Ordonnancement sans ordre : Lorsque 2 tâches sont réalisées sans ordre imposé alors on réalise la même transformation que pour l'alternatif (un espace de travail supplémentaire pour le choix) et dans les espaces de travail des tâches réalisées sans ordre on ajoute un élément d'interaction qui permet d'accéder à l'autre espace de travail.
Une fois les opérateurs d'ordonnancement traités, le caractère itératif des tâches doit être pris en compte pour ajouter des éléments d'interaction permettant de les réaliser plusieurs fois.

Itération de tâche : Un élément d'interaction doit être ajouté au(x) dernier(s) espace(s) de travail atteint pour réaliser la tâche. Cet élément d'interaction doit permettre d'aller vers le(s) premier(s) espace(s) de travail atteint lorsque la tâche débute.
Note : Lorsque la tâche itérative est élémentaire, l'espace origine et l'espace destination sont le même mais il faut tout de même y inclure un élément d'interaction pour exprimer la navigation.
Dans notre exemple, depuis l'espace de visualisation d'une photo il faut mettre un élément d'interaction qui va à l'espace de navigation dans les albums pour modéliser l'itération de la tâche *Regarder une photo*.

Pré-conditions : Sur toutes les transitions (les flèches), vous pouvez ajouter des pré-conditions. Celles-ci sont celles du modèle de tâches.

Dans moodle, une video support permet de voir comment une IHM fil de fer est réalisée sur un cas concret.

Concept Map

Dans l'objectif de structurer l'information sans aborder la manière dont celle-ci va être présentée les concept map sont un autre modèle de représentation. A la différence des IHM Fils de fer, les concept maps ne s'intéressent qu'à la structuration des informations (concepts). Les aspects de navigation entre les groupes de concepts (comme on peut les trouver sous forme d'espaces de travail dans l'IHM fils de fer) ne sont pas représentés dans les concept Maps. Par conséquent, ce modèle rend difficile la structuration des informations au sein d'un même espace. Par exemple sur la figure 4, My Money contient beaucoup de concepts qui pourraient ne pas être présentables sur un seul écran. Aucune recommandation ne pourrait être déduite par le concepteur du Concept Map.

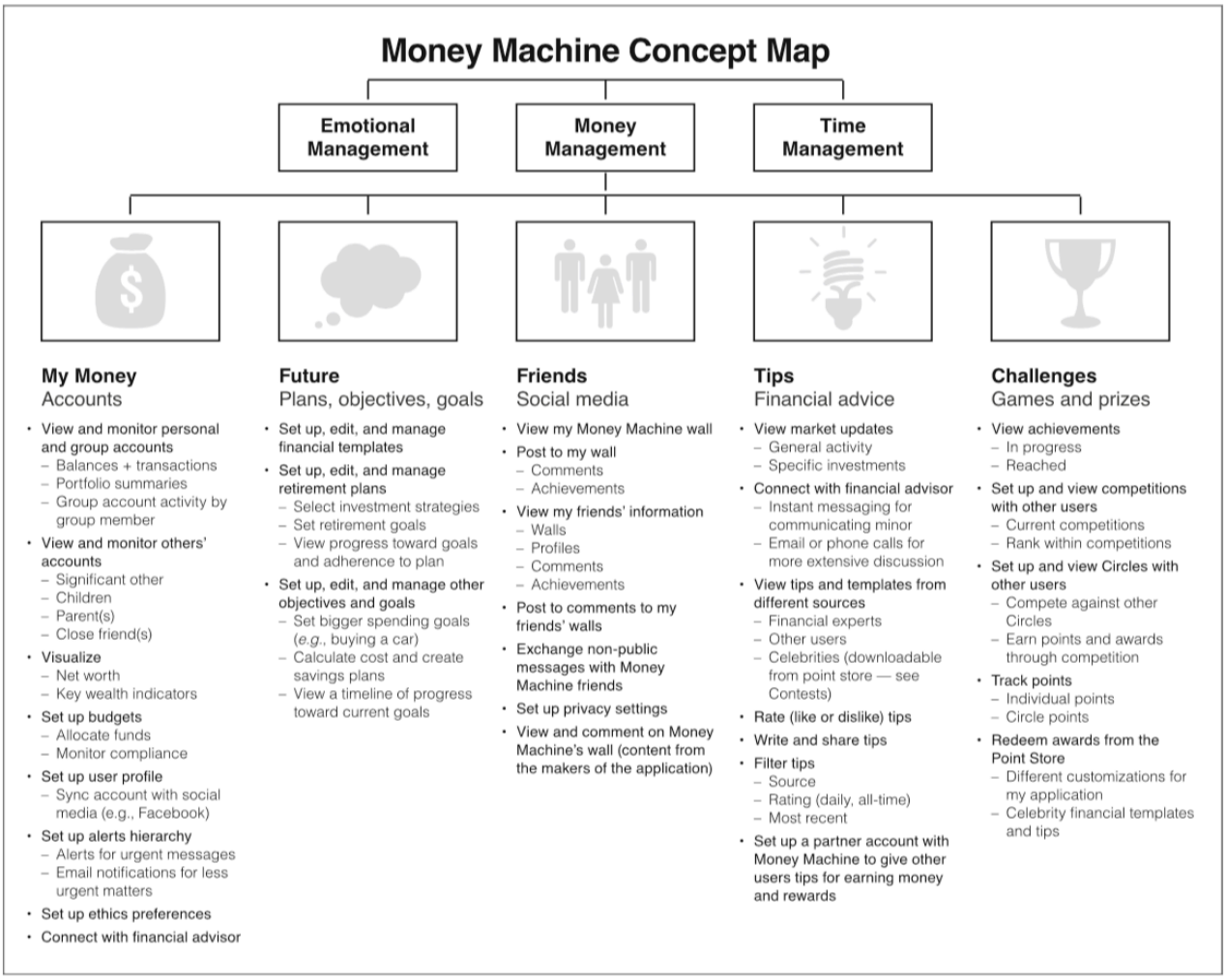


FIGURE 4 : EXEMPLE DE CONCEPT MAP (ISSU DE [MARCUS, 2015])

Exprimer des solutions d'UI à un niveau de concrétisation suffisant (dépendant des modalité d'interaction)

La répartition selon la plateforme : la Meta-IHM

Une méta-IHM permet d'exprimer comment sont réparties les informations en fonction des plateformes.

Par exemple, considérons qu'un seul utilisateur réalise la tâche de la figure 2 dans 3 configurations différentes :

- configuration 1 : utilisation sur un ordinateur de bureau
- configuration 2 : utilisation sur un smartphone
- configuration 3 : utilisation sur un ordinateur de bureau et sur un smartphone (en même temps)

La méta-IHM va alors exprimer quelle.s tâche.s sera réalisée.s sur quelle.s plateforme.s en fonction de la configuration. Si nous reprenons la structuration des informations telle que représentée dans

l'IHM fils de fer, une meta-IHM possible est celle exprimée sur Figure 5.

	Configuration 1	Configuration 2	Configuration 3
Espace de visualisation d'une photo	>80% de la page OV	Page PA	page OV (100%)
Espace de navigation dans les albums	Regroupé dans l'espace de navigation 2 =< 20% de la page OV	Page PN	Page PN
Espace de sélection d'une photo d'un album		Page PNA	Page PNA
Espace de choix du mode de visualisation		Présent sur PN et PA et POD	Présent sur PN et PNA + POV (affichage sur le smartPhone pendant l'affichage de la page OV)
Diaporama	Page OD	Page POD	Page OD

FIGURE 5 : EXEMPLE DE META-IHM POUR LE MODÈLE DE TÂCHES DE LA FIGURE 2

Le comportement dynamique : les modèles de dialogue

Selon la classique définition introduite par le modèle de Seeheim, la dynamique d'une application interactive est contrôlée par la syntaxe du langage d'interaction utilisé. Dans ce modèle, cette fonction est dévolue à un composant logiciel dénommé dialog control, traduit généralement par contrôleur de dialogue. Il est décrit dans [Dix 1991] comme une double liaison :

- d'une part, une liaison avec la sémantique du système interactif pour que celui-ci sache ce qu'il doit faire,
- d'autre part, une liaison avec la présentation pour donner la visualisation de ce système.

Au cœur de l'interaction, le dialogue a été l'objet de nombreuses études. Ainsi, un nombre important de formalismes ont été utilisés pour décrire le dialogue [Olsen 1992]. L'ensemble de ces formalismes peut être classé en trois groupes [Green 1986] :

- les formalismes basés sur les grammaires [Olsen 1992],
- les formalismes à base d'événements [Green 1986] (utilisés dans la plupart des boîtes à outils comme Java/Swing),
- les formalismes basés sur les états tels que les machines à états [Jacob 1985], les statecharts [Harel 1987], ou les réseaux de Petri de haut niveau [Palanque 1992, Sibertin-Blanc 1985].
- Les formalismes basés sur états permettent de représenter graphiquement le comportement du dialogue et favorisent donc le travail de conception.

Les automates

Une machine à états est le formalisme à base d'états qui a, le premier [Jacob 1985], servi à représenter le dialogue d'une application. Depuis, de nombreux modèles à base d'états ont été développés pour exprimer le dialogue des applications interactives.

Les machines à états appelées aussi automates à états ou systèmes de transition étiquetés (STE) sont exprimables sous forme graphique. De plus, elles sont basées sur des modèles mathématiques ; par conséquent, elles permettent le raisonnement et donc, la validation et la vérification de nombreuses propriétés du dialogue dans les IHM telles que les propriétés de sûreté ou de vivacité.

Une machine à états permet de représenter le dialogue d'une application par un ensemble de quatre éléments :

- un ensemble d'états
- un état initial
- un ensemble de transitions
- un ensemble d'étiquettes associée à ces transitions

Les transitions permettent de passer d'un état à un autre et sont activées par des événements tels qu'un mouvement de la souris ou la fin d'une horloge. À chaque transition peuvent être associées des gardes et des actions.

Une garde est une condition de réalisation d'une transition. Elle doit être évaluée à « vrai » pour que la transition soit exécutée. Dans le cas où une garde n'est pas précisée, elle est implicitement vraie.

L'action d'une transition est le code associé à cette transition.

À chaque état, deux actions peuvent être associées, la première nommée action d'entrée, est réalisée lorsque l'état devient l'état courant, et la seconde action de sortie, est exécutée lorsque l'état cesse d'être l'état courant.

Le fonctionnement d'une machine à états se déroule de la manière suivante. Lorsqu'un événement se produit, s'il existe une transition correspondant au traitement de cet événement et partant de l'état courant avec sa garde à vraie, alors les actions : de sortie de l'état courant, de la transition, d'entrée du nouvel état courant sont exécutées. Sinon, l'événement est ignoré.

L'état courant au début de l'exécution de la machine à états est l'état initial.

Un automate peut être représenté graphiquement, chaque état étant représenté par une ellipse et chaque transition par une flèche. L'état initial est, quant à lui, spécifié par une petite flèche sur le côté gauche de l'ellipse. Depuis le milieu des années 2000, les machines à états peuvent être directement implémentées dans l'application grâce à une bibliothèque développée : SwingStates [Appert et Beaudouin-Lafon 2006, Appert et Beaudouin-Lafont 2006]. Cette bibliothèque repose sur une implémentation en JAVA/Swing.

Les machines à états hiérarchiques (Statecharts pour l'IHM)

Le formalisme des machines à états permet une représentation graphique du dialogue d'une application. Cependant, lorsque le dialogue à formaliser est dense, la machine à états produite est d'une taille importante ce qui la rend illisible. Les machines à états hiérarchiques [Blanch 2005] constituent un formalisme qui étend le langage de programmation. Elles font partie de la boîte à outils HsmTk [Blanch 2005], dont l'un des objectifs est de faire en sorte que les interactions deviennent des objets à part entière du vocabulaire du programmeur. Pour atteindre cet objectif, elle utilise un formalisme adapté à la description et à la spécification de l'interaction : les machines à états hiérarchiques.

Les machines à états hiérarchique reposent sur le principe des Statecharts [Harel 1987], permettant de ce fait la définition de machines à états dans les états. C'est de cette structure qu'est issue l'organisation hiérarchique des machines à états entre elles. Elle a donc les composants des machines à états : les états et les transitions.

Le fait d'utiliser le formalisme des Hierarchical State Machine permet de raffiner le comportement d'un composant. Par exemple pour deux versions d'un composant email :

- la première se contente de gérer l'activation ou non d'un email (l'email est-il ou non éditable ?) (Figure 6a).

- la seconde qui, en plus, prend en compte la sélection ou non de l'email dans une liste d'email. L'email est alors « préselectionné » (et donc afficher) mais pas sélectionné, ce qui le rend impossible à éditer (Figure 6b).

La seconde version de l'activation d'un email est donc un raffinement de la première.

La Figure 6 présente les machine à états hiérarchiques des comportements des deux versions du comportement de l'email. La hiérarchisation des états permet de conserver au premier niveau de la machine à états raffinée, les états de la machine initiale. Dans cet exemple, les états Non actif et Actif.

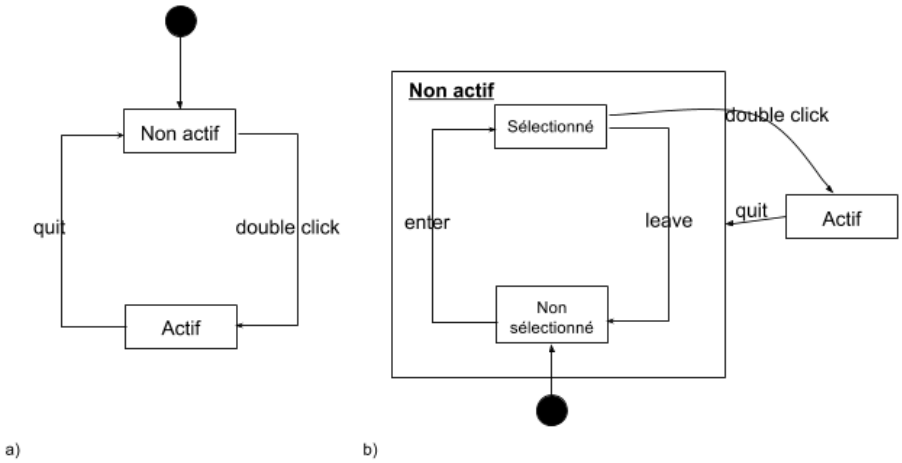


FIGURE 6 : MODÉLISATION SIMPLE ET RAFFINÉE DE LA SÉLECTION D'UN EMAIL

UAN

Il est possible de représenter la dynamique d'une application de manière textuelle. UAN (User Action Notation) définie une grammaire de description à utiliser pour exprimer les actions des utilisateurs sur l'interface, la réaction de l'interface (feedback et modification de l'état) et éventuellement la « connexion » avec le noyau fonctionnel. La figure 7 présente la description avec cette notation de la tâche de suppression d'un fichier.

TASK: delete file			
USER ACTIONS	INTERFACE FEEDBACK	INTERFACE STATE	CONNECTION TO COMPUTATION
~[file_icon] Mv	file_icon-!: file_icon!, ∀file_icon'!: file_icon'-!	selected = file	
~[x,y]*	outline(file_icon) > ~		
~[trash_icon]	outline(file_icon) > ~, trash_icon!		
M^	erase(file_icon), trash_icon!!	selected = null	mark file for deletion

FIGURE 7: DESCRIPTION UAN D'UNE TÂCHE DE SUPPRESSION DE FICHIER ISSUE DE [HARTSON ET AL. 1990]

Action	Meaning
~	move the cursor
[X]	the context of object X, the "handle" by which X is manipulated
~[X]	move cursor into context of object X
~[*x*, *y*]	move the cursor to (arbitrary) point *x*, *y* outside any object
~[*x*, *y* in A]	move the cursor to (arbitrary) point within object A
~[X in Y]	move to object X within object Y (e.g., [OK__icon in dialogue__box])
[X]~	move cursor out of context of object X
v	depress
^	release
Xv	depress button, key, or switch called X
X^	release button, key, or switch X
X^v	idiom for clicking button, key, or switch X
X"*abc*"	enter literal string, *abc*, via device X
X (*xyz*)	enter value for variable *xyz* via device X
( )	grouping mechanism
*	iterative closure, task is performed zero or more times
+	task is performed one or more times
{ }	enclosed task is optional (performed zero or one time)
A B	sequence; perform A, then B (same if A and B are on separate, but adjacent, lines)
OR	disjunction, choice of tasks (used to show alternative ways to perform a task)
&	order independence; connected tasks must all be performed, but relative order is immaterial
⇔	interleavability; performance of connected tasks can be interleaved in time
‖	concurrency; connected tasks can be performed simultaneously
;	task interrupt symbol; used to indicate that user may interrupt the current task at this point (the effect of this interrupt is specified as well, otherwise it is undefined, i.e., as though the user never performed the previous actions)
∀	for all
:	separator between condition and action or feedback
Feedback	**Meaning**
!	highlight object
-!	dehighlight object
!!	same as !, but use an alternative highlight
!-!	blink highlight
(!-!)^n	blink highlight *n* times
@ *x*, *y*	at point *x*, *y*
@ X	at object X
display (X)	display object X
erase (X)	erase object X
X > ~	object X follows (is dragged by) cursor
X ≫ ~	object X is rubber-banded as its follows cursor
outline (X)	outline of object X

FIGURE 8 : LISTE DES SYMBOLES UAN ISSUE DE [HARTSON ET AL. 1990]

Représentations des aspects présentation

Le placement des concepts : Wireframe

Une wireframe peut être plus ou moins concrète, c'est à dire plus ou moins proche de la solution finale et intégrant avec plus ou moins de précision les modalités d'interaction. Cependant, l'objectif des wireframe est bien de considérer le contenu (concept), la position relative des informations et non la forme précise (la présentation du contenu). Le regroupement en fonction de la plateforme (ex : smartphone ou ordinateur de bureau) a déjà été pris en compte. Le détail de l'interaction n'est cependant jamais exprimé dans les wireframe.

La figure 9 présente un exemple de wireframe (déjà très concret) pour l'application Money Machine (même application que pour le concept map de la figure 4).



FIGURE 9 : EXEMPLE DE WIREFRAME (ISSU DE [MARCUS 2015])

Le graphique : la charte graphique

La charte graphique est un document textuel qui présente toutes les directives de présentation graphique de l'interface c'est à dire :

- Le logo
- Les icônes
- Le texte : police, taille, code couleur de chaque type de texte (titre, corps, texte de bouton…)
- Tous les éléments d'interaction (widgets) avec leur taille, forme, couleur, comportement dynamique (que se passe-t-il si le curseur de la souris ?)
- L'organisation spatiale de vos écrans (template) (où se trouve le menu ? Sur quelle dimension ?…)
- Les conséquences d'un redimensionnement de l'écran sur la présentation de celui-ci

Liens entre la présentation et le dynamique : La maquette

Cf chapitre 5

N.B : une fois la charte graphique appliquée et la navigation ajoutée, les maquettes peuvent être légèrement différentes du wireframe. Par exemple, la figure 10 présente la maquette correspondant au wireframe de la figure 9.



FIGURE 10 : EXEMPLE DE WIREFRAME (ISSU DE [MARCUS 2015])